LLM-Guided Grammar-Aware Fuzzing of PostgreSQL

CS-5187 Term Project A3 — Progress meeting 2

Venu Bhuvan Gowtham

26–30 October 2026

Status at a glance

Done since meeting 1

- Every item of the meeting-1 plan implemented and tested on real PostgreSQL master
- 2 previously unknown PostgreSQL bugs, root-caused, with patches
- 12 metamorphic oracle kinds on derivation trees, 18 plan variants, cross-version
- Triage: tree + token reduction, localization, upstream drafts
- Gated LLM arm; 23-bug re-injected benchmark
- Paper drafted (all five graded parts)

Numbers

- 99.5–99.7% valid statements
- 20/23 benchmark bugs observable, only 9 by crashes
- 3,379 checks of new oracle kinds: 0 false positives
- Tree reduction: 116 $\rightarrow$ 79 tokens
- 138 unit tests, $\approx$10.5k lines of Python

Next: overnight campaigns (RQ1–RQ4), file both bugs upstream

Recap: problem and the three claims

Problem. PostgreSQL's planner changes every week. The worst bugs are silent wrong results. Crash fuzzers miss them; SQLancer-style oracles use small hand-written generators; neither knows what changed in yesterday's commit.

1. **One tree, four jobs.** A typed derivation tree generates, mutates, is rewritten by the oracles, and is reduced in triage. **all in code**
2. **Soundness we measure.** Generator validity; per-oracle precision before/after each false-positive class is fixed. **ledger**
3. **The LLM proposes, the server decides.** Fragments and rewrites enter only through a gate on the real server. **gate**

Ladder: Random/Mutation → Grammar → Grammar+Oracles → +LLM, same server, oracles, budget, seeds.

Motivating example: BUG-02 (found by our oracle)

```
SELECT c,
  '(,)'::int4range @> c       AS a, -- want NULL
  'empty'::int4range @> c     AS b, -- want NULL
  NOT ('empty'::int4range @> c) AS d
FROM (VALUES (NULL::int), (1)) v(c);
-- 17, 18.6, master: NULL row ->  t | f | t
```

- Strict operator, but a planner *support function* folds it to `true`/`false` (`rangetypes.c`)
- Silent wrong result in released PostgreSQL 17 and 18
- Found because the oracle knew `'(,)'::int4range` is one typed `expr<int4range>` node: `e` ≡ `(SELECT e)`

PLACEHOLDER

Screenshot: `psql` on 18.6 and master running this query, plus `EXPLAIN VERBOSE` showing `Output: true`

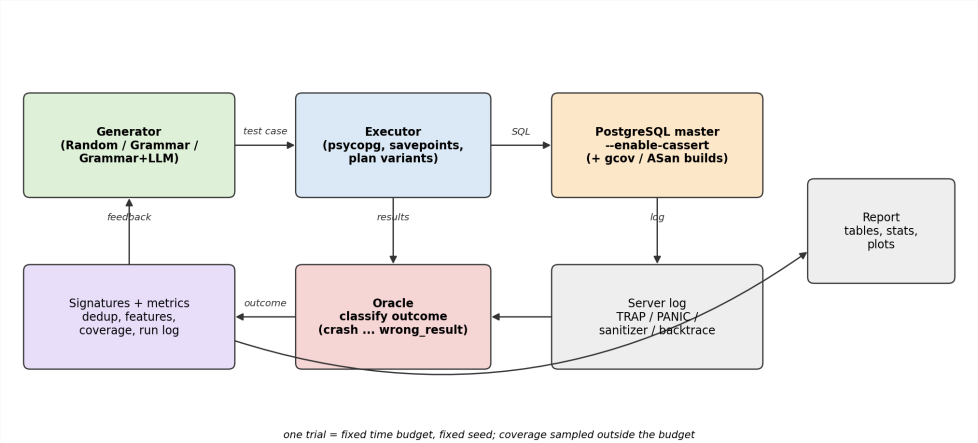
docs/bughunt/BUG-02-*.md

What changed since meeting 1

Plan item	What we built	Status
Metamorphic oracles	TLP (WHERE, HAVING, aggregates), NoREC, expression, predicate, NOT NULL, ORDER BY, CTE, fence	done
Cross-version	master vs 18.6, per-worker DBs, intended-change list	done
Triage	reproduce $\times 3$, tree + token reduction, localization, upstream draft, bisect	done
LLM loop	miner, cached prompt, gate, diff coverage, repair, ledger, rewrite gate	done
Benchmark (RQ3)	23 reverted fixes; fixed / solo / injected servers; attribution	done
Stretch goals	plan-shape guidance, bandit weights, data profile, RLS and CERT oracles	done
Campaigns	RQ1–RQ4 configs; smoke runs pass	running next
Upstream	BUG-01, BUG-02 patches with regression tests	filing this week

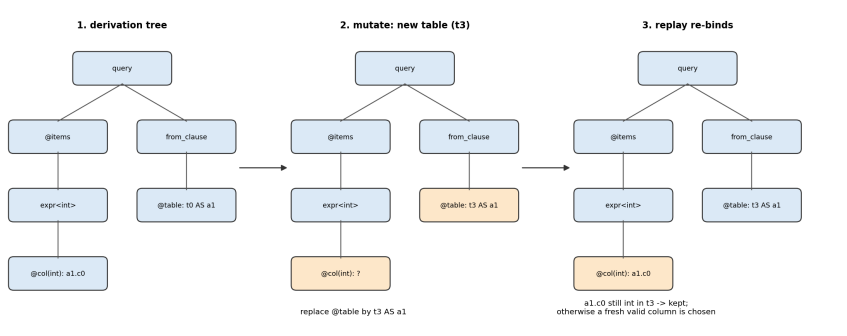
The system

Architecture



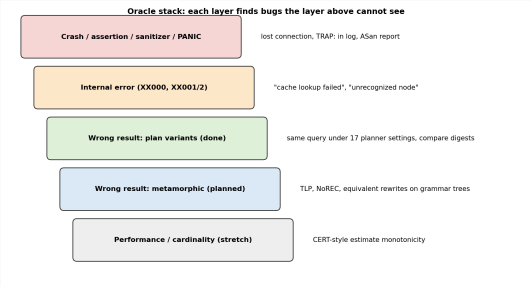
Fuzzing loop (top): derive → attach checks → run several ways → classify/triage. LLM loop (bottom): commit → proposal → gate → grammar.

One tree, four jobs



Generate	typed grammar: 78 rules, 421 alternatives, scopes + type variables
Mutate	regen / splice / insert / drop subtrees; *replay* re-binds names and types
Oracle	rewrite one typed node or clause span $\rightarrow$ equivalent query
Reduce	replace subtrees by their smallest derivation; replay keeps SQL valid

The oracle stack



- **Crash / assertion / internal error** on every statement (cassert build, log + PID check)
- **18 plan variants**: join methods, hash agg, parallel, partition-wise, tiny `work_mem`, self-join elim., eager agg, JIT, ...
- **12 metamorphic kinds** on the tree
- **Cross-version**: master vs 18.6
- Determinism filter, canonical digests (numeric scale, −0, float classes), confirmation re-run

Metamorphic rewrites on derivation trees

TLP-WHERE (generated + rewritten by the tool)

```
SELECT t0_1.c0 FROM t0 AS t0_1
  ==
(SELECT .. WHERE (t0_1.c0 > 5))
UNION ALL (SELECT .. WHERE NOT (t0_1.c0 > 5))
UNION ALL (SELECT .. WHERE (t0_1.c0 > 5) IS
    NULL)
```

Also: TLP-HAVING, TLP-aggregates (MIN of MINs, SUM of COUNTs), NoREC, `IN`↔`OR`, a<b↔`NOT(a>=b)`, `BETWEEN`, NOT NULL conjunct, drop `ORDER BY`, CTE (not) materialized, `OFFSET 0` fence, typed expression identities.

Why the tree matters

The tree knows *which* `WHERE` belongs to which `SELECT`, each node's type, grouped levels, scopes and outer joins. So each rewrite is applied only where it is sound:

- multiset contexts only
- `DISTINCT`: unique-text types only
- NOT NULL: never under outer joins
- `e+0` on `int`, not on `float8`

Beyond wrong results: RLS leak oracle and CERT

RLS leak oracle (information flow, L21–22)

- Policies on 5 tables, role `pgfuzz_rls` (sees 131 of 203 rows of `t0`)
- Left: query as the restricted role
- Right: query as superuser with each table replaced by `(SELECT * FROM t WHERE policy)`
- More rows on the left = leak
- 532 checks, 0 false positives

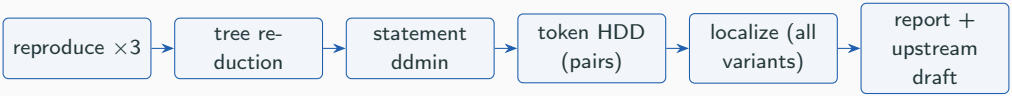
CERT (Ba & Rigger, ICSE'24)

- Adding `WHERE` must not *raise* the planner's row estimate
- Outcome `performance`, reported apart from wrong results
- 58 checks, 0 violations

Cross-version

1,155 comparisons, 0 mismatches after fixing clock data and excluding intended changes (`corr` on master)

Triage: from failure to upstream report



BUG-01 case (590 tokens)	tokens	runs
token reduction only	116	406
tree first, then tokens	**79**	555

- Localization: BUG-02 → "not toggled by planner settings" → support function (correct)

PLACEHOLDER

Screenshot: `pgfuzz triage` output and the generated `upstream_report.txt`

results/triage/<signature>/

The LLM arm: the LLM proposes, the server decides

1. Mine master commits (planner, executor, parser), excluding the 23 benchmark fixes
2. Cached 9k-token prompt + commit digest → fragment, seeds, rewrites (JSON schema)
3. **Gate**: parse · derivable ($\geq$5 uses) · valid on server ($\geq$60%) · seeds run · diff coverage up
4. Repair rounds with the server's errors (max 3); ledger of tokens and cost

API or manual mode; stored responses replay for free.

Rewrite gate (value probe + 150 live test cases)

Proposed rewrite	Result
`CASE WHEN e IS NULL THEN NULL ELSE e END`	accepted, 90/90
`substr(e, 1)`	accepted, 88/88
`e + 1`	rejected: 12 mismatches
`e IS TRUE`	rejected: probe at NULL

`e IS TRUE` passed 109 sampled checks: sampling is not proof.

Evaluation

RQ3 benchmark: 23 real bugs put back into PostgreSQL

- Fixes from 31 Jul–16 Oct 2026 with regression tests, reverted on base `26bd08fb`
- Servers: **fixed**, **solo**$_i$ (one bug), **injected** (all 23)
- `bench verify`: symptom = fixed vs solo difference
- `bench attribute`: replay finding on each solo server

Key fact

20/23 observable, only 9 visible to a crash-only fuzzer.

Symptom	Bugs
wrong result	8
internal error	5
assertion	4
missing / wrong error	3
not observable (excluded)	3

PLACEHOLDER

Screenshot: `bench verify` table (fixed / solo / injected per bug)

src/bench/reinjected/verify.json

Smoke run 2: 15 min per arm on the injected server

Arm	Test cases	Valid	Features	Plan shapes	Bench bugs
Random/Mutation	174,104	26.3%	17	22	0
Grammar	16,841	99.6%	101	5,416	0
Grammar+Oracles	15,861	99.7%	101	5,968	1 (158 s)
+LLM (no fragments yet)	15,800	99.7%	101	5,937	1 (160 s)

- 10× more test cases do not buy reach: random reaches 17 features
- `rb-dfcf6fa7` found only with oracles + plan guidance; attribution by replay separated it from a bug with the same message
- One trial each: this shows the pipeline works, *not* a significant difference

Component checks on master (single runs)

Component	Measured
Metamorphic oracles (first run)	907 checks, 0 FP; BUG-02 in the first 150 test cases
New rewrite kinds	798 checks, 0 false positives
Full system, 3 min	3,166 test cases, 99.5% valid, 1,991 checks, 7,956 variant runs, 0 FP
Bandit weights	533 vs 419 plan shapes per 1,000 test cases, same validity
Data profile (`@dlit`)	selective predicates (5–95% of rows): 29.3% $\rightarrow$ 58.6%
Tree replay	599/600 stored trees replay to identical SQL
Token reduction (BUG-02)	4 statements / 107 tokens $\rightarrow$ 1 / 36 in 307 runs
LLM fragment gate	valid fragment 251/251 accepted; invalid rejected with server message

Precision ledger (RQ2, claim 2)

Oracle	Cand.	Real	FP	After fixes
assertion	2	1	1	1.00
crash	1	0	1	–
cross-version	3	0	2	–
internal error	2	1	0	1.00
mm: expr	1	1	0	1.00
mm: tlp_having	4	0	4	–
plan-variant	5	0	5	–
all	18	3	13	1.00

2 flaky. Raw precision 0.19, logged on purpose.

6 false-positive classes, all fixed + tested

- `ARRAY(SELECT)` without `ORDER BY`
- TLP-HAVING without `GROUP BY`
- numeric scale under `DISTINCT`
- clock in seed data across versions
- our PoC replay crashed a shared server → PID check
- intended `corr` change on master

Lesson: oracle precision, not generator power, is the bottleneck.

Bugs found

BUG-01 — JSON_TABLE collations

- Found by the grammar campaign (assertion, `execExpr.c:2078`)
- 18.6: `ROW(1,2) < ROW(3,4)` in the context item returns false
- Missing `assign_expr_collations()` in `parse_jsontable.c`
- One-line patch + regression tests; 17, 18, master

BUG-02 — range containment w/ NULL

- Found by `expr:scalar_sub` rewrite
- `find_simplified_clause()` folds to a constant
- Patch: simplify only if the element is provably non-null
- `make check` passes; 17, 18, master

PLACEHOLDER

Upstream status: pgsql-bugs numbers + thread links for BUG-01 and BUG-02

postgresql.org/list/pgsql-bugs

Experiment design (RQ1–RQ4)

RQ1 reach: validity, line/branch coverage, features, plan shapes

RQ2 bugs found and per-oracle precision

RQ3 re-injected bugs found, time to find

RQ4 what the LLM adds (diff coverage, ablation) and costs

- RQ1: 7 arms (incl. no-bandit and LLM ablations) $\times$ 5 trials $\times$ 1 h, gcov build
- RQ3: 4 arms $\times$ 5 $\times$ 1 h, injected server + 18.6 reference
- Paired seeds, same oracles and budget; one commit (`26bd08fb`) for everything
- Medians, IQR, bootstrap 95% CI, Mann–Whitney U, Vargha–Delaney $\hat{A}_{12}$

RQ1 results (to fill after the campaign)

PLACEHOLDER

Plot: backend line coverage over time, median and IQR band per arm

pgfuzz report ../results/rq1 → plots/

PLACEHOLDER

Table: validity, coverage, features, plan shapes per arm with p and $\hat{A}_{12}$

results/rq1/report.md

RQ3 and RQ4 results (to fill after the campaigns)

PLACEHOLDER

Table: bugs found per arm (out of 20), time to first detection per bug

pgfuzz bench attribute → table_rq3.tex

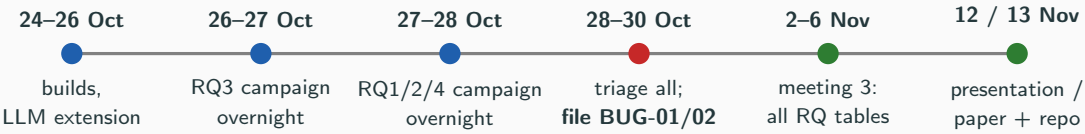
PLACEHOLDER

LLM ledger: fragments accepted/rejected, repair rounds, diff coverage with/without, tokens and $

pgfuzz llm report

Plan

Timeline to the deadline



- Exact commands and expected times in `docs/EXECUTION_PLAN.md` (phases 0–5, ≈16 h of machine time, ≈1.5 h of ours)
- Campaigns resume if interrupted

Risks and what we would like feedback on

Risks and fallbacks

- New false-positive classes in long runs $\rightarrow$ triage; they are RQ2 data
- LLM API unavailable $\rightarrow$ manual mode, replayed responses
- No upstream reply by 13 Nov $\rightarrow$ report filing, reproducer, patch
- 5 trials may not reach $p < 0.05$ $\rightarrow$ effect sizes + CIs; more trials if time

Questions for you

- Is 5 trials $\times$ 1 h per arm acceptable, or should we cut arms for 10 trials?
- Does a filed bug with a patch, under review by 13 Nov, count towards the top band?
- Paper is $\approx$1 page over 12: cut related work or component checks?

Live demo (backup screenshots)

PLACEHOLDER

`pgfuzz sample -execute`: generated SQL and 99.5% validity

terminal

PLACEHOLDER

`pgfuzz triage`: 590 → 79 tokens, report

docs/evidence/

PLACEHOLDER

RQ3 smoke report: medians, $\hat{A}_{12}$, time to first bug

rq3_smoke2/report.md

Thank you — questions?

github.com/v28k/cs5187-term-project